

Labo 00 — Praktijklabo: een rondleiding door Linux via de terminal

Doel: elk begrip uit de theorie manipuleren — processen, signalen, exitcodes, permissies, omgeving, stromen, poorten, archieven — met niets anders dan wat Ubuntu 24.04 standaard meeleverd. Geen enkele container hier: alles wat je ziet komt, exact zo, terug in de Docker-labo's.

Vereisten — Windows 10/11 met WSL 2 en een **Ubuntu 24.04**-distributie. Niets anders: geen Podman (labo 01), geen extra pakket. Open een Ubuntu-terminal en blijf erin.

Stap 0 — Waar ben ik, en wie ben ik?

```
head -n 2 /etc/os-release
uname -r
whoami
id
```

Observeer `PRETTY_NAME="Ubuntu 24.04.x LTS"`, een kernel `6.6.87.2-microsoft-standard-WSL2` (het achtervoegsel is de WSL-handtekening), je gebruikersnaam, en een regel `uid=1000(...)` `gid=1000(...)` `groepen=... 27(sudo) ...`.

Uitleg. Drie identiteiten om nooit meer te verwarren: de **distributie** (Ubuntu 24.04, de userland), de **kernel** (door Microsoft gecompileerd voor WSL), en **jij** (UID 1000, lid van de groep `sudo`). De kernel zal van jou alleen dat getal kennen: 1000.

→ WINDOWS / WSL

Als `uname -r` geen `-microsoft-standard-WSL2` toont, zit je niet in WSL 2 (`wsl --version` en `wsl --list --verbose` aan PowerShell-kant om te controleren). De hele laboreeks veronderstelt WSL 2.

Stap 1 — De kernel, de userland, en de grens

```
cat /proc/version
type ls
type cd
which cat
```

Observeer: de kernelversie voluit; `ls` is `/usr/bin/ls` (een programma, een bestand op schijf); `cd` is a shell builtin (geen programma: een interne functie van de shell); `/usr/bin/cat`.

Uitleg. Alles wat je typt is ofwel een **programma** uit de userland (een uitvoerbaar bestand ergens), ofwel een intern shellcommando. Geen van beide raakt de hardware aan: ze passeren via de system calls van de kernel. `cd` is intern om een precieze reden: van map veranderen is een attribuut *van het shellproces zelf* — een extern programma zou de map van zijn eigen proces veranderen en dan sterven, zonder effect op jou.

→ LINUX

`type` ondervraagt de shell ("wat zou jij met dit woord doen?"); `which` zoekt alleen in het `PATH`. Bij twijfel over een commando dat "liegt" (alias, functie) zegt `type` altijd de waarheid.

Stap 2 — Eén boom, meerdere mounts

```
ls /  
findmnt /  
df -h /  
ls /mnt/c/Windows 2>/dev/null | head -n 3
```

Observeer de unieke root (`bin boot dev etc home ... proc ... tmp usr var`), de `findmnt`-regel van het type `/ /dev/sdc ext4 rw,relatime,...`, een `df` in de orde van `1007G` (de *virtuele* grootte van de WSL-schijf), en — dit is Windows gezien vanuit Linux — de inhoud van `C:\Windows`.

```
findmnt -t proc  
ls /proc | head -n 8  
grep MemTotal /proc/meminfo
```

Observeer `/proc proc proc rw,relatime`: een bestandssysteem van het type `proc`, zonder schijf erachter. De `ls` toont **getallen** — één per levend proces — en `MemTotal` komt rechtstreeks van de kernel.

Uitleg. Geen stations `C:` / `D:`: alles hangt aan dezelfde boom via **mounts**. De Linux-schijf levert `/`, de Windows-schijf is gemount op `/mnt/c`, en de kernel zelf is gemount op `/proc` — een map waarvan de bestanden bij elke lezing ter plekke worden gefabriceerd. Docker-images (labo 02) en volumes (labo 06) zullen alleen maar mounts aan deze boom toevoegen.

→ WINDOWS / WSL

`/mnt/c` steekt een grens Windows ↔ Linux over: dat is **traag**. Een project dat je compileert of images die je opslaat moeten aan de Linux-kant leven (`/home/...`), niet in `/mnt/c/Users/...`. Een reflex om nu al aan te leren.

Stap 3 — Processen: PID, ouder, /proc

```
ps  
echo $$  
ps -p 1 -o pid,comm
```

Observeer: een bijna lege `ps` (jouw `bash`, de `ps` zelf); het PID van je shell (`echo $$`); en proces 1: `systemd`. `ps` alleen toont enkel de processen van *jouw terminal*; alle andere (`ps -ef` toont ze) draaien zonder terminal — voor het merendeel zijn dat **daemons**, dienstprocessen zoals `systemd` zelf, met een naam die vaak op "d" eindigt.

Start nu een proces dat blijft duren, op de achtergrond:

```
sleep 300 &  
ps -o pid,ppid,stat,cmd
```

Observeer een regel `sleep 300` waarvan het **PPID het PID van jouw bash is**: je zag zonet een afstamming.

PID	PPID	STAT	CMD
2363	2362	S	bash
2419	2363	S	sleep 300
2420	2363	R	ps -o pid,ppid,stat,cmd

Ga dat proces bekijken in `/proc` (vervang `2419` door jouw PID):

```
head -n 3 /proc/2419/status
tr '\0' ' ' < /proc/2419/cmdline; echo
ls -l /proc/2419/exe
```

Observeer `Name: sleep`, `State: S (sleeping)`, de exacte commandoregel, en een link `exe -> /usr/bin/sleep`.

Uitleg. `ps` heeft niets magisch: het leest `/proc`. Alles wat Docker je later toont (`podman top`, `podman inspect`) komt daar ook vandaan. `STAT S` betekent *sleeping* — wachtend; `R`, *running*.

Stap 4 — Signalen en exitcodes

De `sleep` draait nog. Stuur hem beleefd weg:

```
kill 2419          # jouw eigen PID
ps -o pid,cmd | grep "[s]leep 300" || echo "geen sleep-proces meer"
```

Observeer `Terminated` (getoond door de shell) en dan `geen sleep-proces meer`: `kill` zonder optie stuurt `SIGTERM`, en `sleep` gehoorzaamt.

Opnieuw, maar brutaal:

```
sleep 300 &
kill -9 %1
```

Observeer deze keer `Killed`: `SIGKILL` heeft niets gevraagd. (`%1` verwijst naar *job* nr. 1 van de shell — handig om het PID niet te moeten opzoeken.)

Nu de verzameling exitcodes:

```
true; echo $?
false; echo $?
ls /bestaat-niet; echo $?
onbekend-commando; echo $?
bash -c 'kill -9 $$'; echo $?
```

Observeer, in volgorde: `0`, `1`, `2` (na de foutmelding van `ls`), `127` (na `command not found`), en `137` (na `Killed`).

Uitleg. `0` = succes, de rest = mislukking, en `128 + n` = dood door signaal *n*: `137 = 128 + 9` = gedood door `SIGKILL`. Deze vijf getallen zijn exact wat `podman ps` in zijn kolom `Exited (...)` zal tonen in labo 03 — leer ze hier lezen, waar alles eenvoudig is.

ONTHOUDEN

De beschaaftde escalatie: `kill` (SIGTERM, de applicatie mag opruimen), wachten, en pas dan `kill -9` (SIGKILL, de kernel wist). `docker stop` past dat protocol automatisch toe: SIGTERM, 10 seconden gratie, SIGKILL.

Stap 5 — De omgeving en het PATH

```
echo $HOME
env | wc -l
env | grep -E '^(HOME|PATH|LANG)='
```

Observeer je omgeving: enkele tientallen variabelen, waaronder `HOME=/home/<jij>` en een `PATH` dat, op WSL, ook Windows-paden bevat (`/mnt/c/Windows/system32` ...).

Het beslissende experiment — een shellvariabele is **geen** omgevingsvariabele:

```
MSG=hallo
echo $MSG
bash -c 'echo kind ziet: [$MSG]'
export MSG
bash -c 'echo kind ziet: [$MSG]'
```

Observeer: `hallo`, dan `kind ziet: []` (leeg!), en dan, na `export`, `kind ziet: [hallo]`.

Uitleg. Elk kindproces krijgt een **kopie** van de omgeving van de ouder, bevroren bij de start. Vóór `export` bestond `MSG` alleen in jouw shell. Dit exacte mechanisme gebruikt `podman run -e MSG=hallo` in labo 08 om je applicaties te configureren.

Vervolgens het `PATH`:

```
mkdir -p ~/labo0/tools
printf '#!/bin/bash\necho "eigen tool: ok"\n' > ~/labo0/tools/mijntool
chmod +x ~/labo0/tools/mijntool
mijntool; echo $?
export PATH="$HOME/labo0/tools:$PATH"
mijntool
```

Observeer eerst `command not found` en `127`, en dan, zodra de map aan het `PATH` is toegevoegd, `eigen tool: ok`.

Uitleg. De shell "kent" geen enkel commando: hij zoekt een uitvoerbaar bestand met die naam in de mappen van het `PATH`, in volgorde, en stopt bij het eerste dat hij vindt. (Dit gewijzigde `PATH` geldt alleen voor deze shell; permanent = één regel in `~/.bashrc`.)

Stap 6 — Drie stromen: redirections en pipes

```
cd ~/labo0
echo "eerste regel" > notes.txt
echo "tweede regel" >> notes.txt
cat notes.txt
```

Observeer: `>` maakt aan (of overschrijft!), `>>` voegt toe.

Scheid nu de twee uitvoerstromen:

```
ls notes.txt /bestaat-niet > uitvoer.txt 2> fouten.txt
cat uitvoer.txt
cat fouten.txt
```

Observeer: het scherm bleef stil tijdens de `ls`; `uitvoer.txt` bevat `notes.txt`, `fouten.txt` bevat `ls: cannot access '/bestaat-niet': No such file or directory`.

```
ls notes.txt /bestaat-niet > alles.txt 2>&1
cat alles.txt
```

Observeer de twee regels samen: `2>&1` sluit de foutstroom (2) aan op waar de uitvoer (1) naartoe wijst.

Ten slotte de pipes:

```
ps -ef | wc -l
ps -ef | grep "[b]ash" | head -n 3
```

Observeer het aantal processen van het systeem, en dan jouw shells — zonder tussenbestand: de uitvoer van elk commando voedt de invoer van het volgende.

→ LINUX / SHELL

De truc `grep "[b]ash"`: de haakjes vormen een reguliere expressie die `bash` matcht... maar de regel van de `grep` zelf bevat `[b]ash`, dat zichzelf niet matcht. Zonder dat zou `grep` altijd zichzelf vinden. Je ziet dit patroon in alle labo's.

Stap 7 — Permissies: een `ls -l` lezen

```
ls -l notes.txt
stat -c "%U %G %a %n" notes.txt
```

Observeer `-rw-r--r-- 1 <jij> <jij> 27 ... notes.txt` en de numerieke vorm `644`: eigenaar `rw` (6), groep `r` (4), anderen `r` (4).

Maak een script en probeer het uit te voeren:

```
printf '#!/bin/bash\nnecho "Hallo, ik ben proces $$"\n' > hallo.sh
./hallo.sh; echo $?
chmod +x hallo.sh
ls -l hallo.sh
./hallo.sh
```

Observeer: `Permission denied` en code **126** (gevonden maar niet uitvoerbaar); daarna, na `chmod +x`, `-rwxr-xr-x` en het script dat draait — met bij elke start een ander PID.

En de root-grens:

```
cat /etc/shadow; echo $?
ls -l /etc/shadow
sudo head -n 1 /etc/shadow
```

Observeer `Permission denied` (code 1), de regel `-rw-r----- 1 root shadow ...` die het verklaart (je bent noch `root` noch van de groep `shadow`), en dan, via `sudo`, de eerste regel `root*:...` (`*` of `!`: vergrendeld account, geen enkel wachtwoord aanvaard).

Uitleg. De kernel vergelijkt de UID van het proces met de drie `rwX`-tripletten en past het eerste toe dat op jou slaat. `sudo` "omzeilt" niets: het start het proces met UID 0, waaraan de kernel niets weigert. In labo 06, wanneer een container in een volume bestanden schrijft die aan een onverwachte UID toebehoren, is dit het leesrooster dat je nodig hebt.

Stap 8 — Een server, een poort, een client

Ubuntu 24.04 levert Python mee: je eerste HTTP-server in één regel.

```
echo "<h1>Hallo vanaf mijn server</h1>" > index.html
python3 -m http.server 8080 &
curl -s http://localhost:8080/index.html
```

Observeer je HTML teruggestuurd via HTTP: `<h1>Hallo vanaf mijn server</h1>`.

```
curl -si http://localhost:8080/index.html | head -n 4
ss -tlnp | grep 8080
```

Observeer het volledige HTTP-antwoord (`HTTP/1.0 200 OK`, `Server: SimpleHTTP/0.6 Python/3.12.3`, `Content-type: text/html`) en de luisterregel:

```
LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:(("python3",pid=2788,fd=3))
```

`0.0.0.0:8080`: het proces `python3` luistert op **alle** interfaces, poort 8080.

→ **WINDOWS / WSL**

Open een **Windows**-browser op `http://localhost:8080`: de pagina verschijnt. WSL 2 stuurt `localhost` automatisch door van Windows naar Ubuntu. Die doorschakeling laat je in labo 07 toe om je containers te testen vanuit een Windows-browser.

Probeer nu een geprivilegieerde poort:

```
python3 -m http.server 80
```

Observeer de mislukking: `PermissionError: [Errno 13] Permission denied`. Poort 80 ligt onder de drempel 1024, voorbehouden aan root — en jij bent UID 1000. Podman rootless erft dezelfde limiet.

Zet de server uit en controleer:

```
kill %1  
curl -s --max-time 2 http://localhost:8080/; echo $?
```

Observeer code **7** van `curl`: *connection refused* — niemand luistert nog.

Stap 9 — Archiveren: `tar`, de voorouder van de images

```
mkdir -p mijn-app/config  
echo "app.port=8080" > mijn-app/config/app.properties  
echo "namaakbinary" > mijn-app/app.bin  
tar -czf mijn-app.tar.gz mijn-app  
ls -lh mijn-app.tar.gz  
file mijn-app.tar.gz
```

Observeer een archief van enkele honderden bytes, geïdentificeerd als `gzip compressed data`.

```
tar -tf mijn-app.tar.gz  
mkdir -p /tmp/herstel  
tar -xzf mijn-app.tar.gz -C /tmp/herstel  
cat /tmp/herstel/mijn-app/config/app.properties
```

Observeer de inhoudslijst (`-t` = *test/list*), dan de extractie elders (`-C`) en het identiek herstellde bestand: `app.port=8080`.

Uitleg. `tar` (*tape archive*, 1979) stopt een volledige boomstructuur — paden, permissies, eigenaars — in één bestand. Onthoud het goed: een **layer** van een Docker-image is letterlijk een tar-archief, en `podman save` (labo 02) levert je een tar van tars. Niets nieuws onder de zon.

Opruimen

Controleer dat er geen laboproces meer rondhangt, en verwijder dan de bestanden:

```
ps -o pid,cmd | grep -E "[s]leep|[h]ttp.server" || echo "niets te doden"  
rm -r ~/labo0  
rm -r /tmp/herstel
```

Het gewijzigde `PATH` en de variabele `MSG` verdwijnen met deze shell: sluit de terminal. (Er werd niets geïnstalleerd: er valt niets te deïnstalleren.)

Wat je nu moet kunnen beweren

- Mijn kernel is `...-microsoft-standard-WSL2`; mijn distributie is Ubuntu 24.04; ik ben UID 1000.
- Een proces heeft een PID en een ouder; ik zag het geboren worden (`&`), leven (`/proc/<pid>/`) en sterven (`kill`).
- `kill` stuurt SIGTERM (onderhandelbaar), `kill -9` SIGKILL (niet onderhandelbaar); een proces gedood door SIGKILL eindigt met `137` = `128 + 9`.
- `$?` is `0` bij succes; `126` = niet uitvoerbaar, `127` = niet gevonden in het `PATH`.
- Een variabele bereikt kindprocessen pas na `export` — en nooit al gestarte processen.
- `>` vangt stdout, `2>` stderr, `2>&1` voegt ze samen, `|` schakelt processen aan elkaar.
- `-rw-r-----` lees je in drie tripletten; de kernel vergelijkt UID's, en `root` (UID 0) negeert het rooster.
- `ss -tlnp` zegt me wie op welke poort luistert; `0.0.0.0` = alle interfaces; `< 1024` = alleen root; `curl` test het geheel.
- Een mount haakt een bestandssysteem aan de unieke boom; `/proc` heeft geen schijf; `tar` verpakt een boomstructuur — Docker-images doen straks hetzelfde.